

Operations on Processes

1. Process Creation:

- **Forking:** Creating a new process by duplicating an existing one, often used in Unix-based systems. The new process is an identical copy of the original (parent) process.
- **Executing a Program:** After forking, the child process usually executes a different program using functions like `exec()` in Unix-based systems, replacing its memory space with the new program's code and data.

2. Process Termination:

- **Exiting:** A process can end its execution voluntarily and release its resources using a system call like `exit()`. The operating system performs necessary cleanup after the process exits.
- **Abnormal Termination:** A process may terminate abnormally due to errors, violations, or exceptions, requiring the operating system to handle the situation and clean up resources.

3. Process Scheduling:

- **CPU Scheduling:** Deciding the order in which processes run on the CPU, typically managed by scheduling algorithms like round-robin or priority-based scheduling. The scheduler determines which process gets CPU time.

4. Process States:

- **Running:** The process is actively executing instructions on the CPU.
- **Ready:** The process is prepared to execute and waiting for its turn on the CPU.
- **Blocked (or Waiting):** The process is unable to execute due to an event (e.g., I/O operation) and is waiting for that event to occur.

5. Inter-Process Communication (IPC):

- **Message Passing:** Processes communicate by sending and receiving messages through message queues or other IPC mechanisms, allowing them to share information and coordinate actions.
- **Shared Memory:** Processes share a portion of memory, enabling communication by reading and writing to this shared memory region.

6. Process Control Block (PCB):

- Each process is represented by a PCB, holding vital process information like process ID, program counter, register values, memory allocation, file descriptors, and scheduling details.
- The operating system uses the PCB to manage and control processes efficiently, making context switches and scheduling decisions.

Scheduler

In an operating system, a scheduler is a crucial component responsible for determining the order and priority of process execution on the CPU. It allocates CPU time to processes efficiently, ensuring fair utilization of system resources. The scheduler is a fundamental part of multitasking systems, allowing for the simultaneous execution of multiple processes.

Types of Schedulers

1. Long-Term Scheduler (Job Scheduler):

- Determines which processes from the pool of new processes should be loaded into memory for execution.
- Controls the degree of multiprogramming, deciding how many processes can be in the ready queue at a time.

2. Short-Term Scheduler (CPU Scheduler):

- Selects processes from the ready queue (those already in memory) and allocates the CPU to them.
- Decides the order and priority of process execution, executing the chosen process on the CPU.

3. Medium-Term Scheduler:

- Swaps processes in and out of main memory (RAM) and secondary storage (e.g., disk), managing the degree of multiprogramming.

4. Admission Control Scheduler:

- Determines whether a new process should be admitted to the system based on system load, available resources, and system capacity.

Context Switching

Context switching is a vital operating system mechanism that allows the system to efficiently manage and switch between multiple processes or threads running concurrently on a CPU core. It involves saving the context (state) of a running process and restoring the context of another process, enabling multitasking and the illusion of parallel execution.

Purpose of Context Switching:

- **Multiprogramming:** Enables running multiple processes in a time-shared manner on a single CPU core, enhancing system throughput and responsiveness.
- **Concurrency:** Allows the operating system to provide the appearance of parallel execution by frequently switching between processes.

Key Components of a Process Context:

- **Program Counter (PC):** Holds the address of the next instruction to be executed in the process.
- **Register Values:** Contents of CPU registers (e.g., general-purpose registers, program counter, stack pointer) that define the current state of the process.
- **Stack and Stack Pointer:** The process's call stack, used for function calls and local variables.
- **Process Control Block (PCB) Information:** Process ID, state, scheduling information, memory management details, etc.

Process of Context Switching:

1. Save Current Context:

- The operating system saves the current context (register values, PC, etc.) of the running process being preempted. This information is stored in the process's PCB.

2. Select Next Process:

- The scheduler selects the next process to execute from the ready queue or another scheduling structure based on scheduling policies and priorities.

3. Load Next Context:

- The saved context of the selected process (from its PCB) is loaded into the CPU registers and program counter, preparing it for execution.

4. Resume Execution:

- Execution of the new process begins from where it left off, continuing seamlessly as if it were never interrupted.

Threads in Operating Systems

Threads are the smallest unit of execution within an operating system. A thread is a sequence of instructions that can be scheduled and executed independently. Threads within a process share the same memory space, file handles, and other resources, but each thread has its own stack and program counter.

Threads enable concurrent execution of tasks, allowing an application to perform multiple operations simultaneously and improve system efficiency.

Thread Models in Operating Systems

1. Many-to-One Model (M:1)

- Many user-level threads map to a single kernel-level thread.
- Thread management is handled by the application or a runtime library at the user level without operating system support.
- Efficient in terms of thread creation and management, but lacks parallelism as only one thread can execute at a time.

2. One-to-One Model (1:1)

- Each user-level thread corresponds to a separate kernel-level thread.
- Provides concurrency and parallelism, allowing multiple threads to run in parallel on multicore systems.
- Offers good performance but can be inefficient in terms of thread creation and management due to the overhead of managing individual kernel-level threads.

3. Many-to-Many Model (M:N)

- Many user-level threads are multiplexed onto a smaller or equal number of kernel-level threads.
- Allows for both parallelism and concurrency.
- Provides flexibility in managing thread-to-kernel mappings to optimize performance and resource utilization.

4. Two-Level Model (2-level)

- Combines aspects of both the one-to-one and many-to-many models.
- Allows multiple user-level threads to be mapped to a smaller number of kernel-level threads.
- Provides a balance between performance and thread management efficiency.

User-Level Threads

User-level threads (ULTs) are threads managed entirely by the application or a user-level thread library. The operating system is unaware of their existence and does not provide direct support for their management. Here are key points about user-level threads:

- **Thread Management:** Creation, scheduling, and synchronization of user-level threads are handled at the application level without kernel involvement. The operating system views the entire process as a single thread of execution.
- **Efficiency and Performance:** User-level threads are typically more efficient in terms of thread management overhead, as the application manages the thread lifecycle directly. Thread creation, switching, and synchronization can be faster since they don't involve kernel calls.
- **Drawbacks:**
 - **Blocking:** If one thread in a process blocks (e.g., due to I/O), it blocks the entire process, including other threads within it.
 - **Limited Concurrency:** User-level threads cannot fully utilize multicore/multiprocessor systems, as a single kernel-level thread is associated with the entire process.

Kernel-Level Threads

Kernel-level threads (KLTs) are threads managed and scheduled directly by the operating system's kernel. Here are key points about kernel-level threads:

- **Thread Management:** The operating system kernel is responsible for creating, scheduling, and managing kernel-level threads. Each thread is an independent entity as far as the kernel is concerned.
- **Efficiency and Performance:** Kernel-level threads provide better concurrency, parallelism, and responsiveness since the kernel can schedule threads independently on multiple CPU cores. However, the management overhead and system call costs are higher compared to user-level threads.
- **Advantages:**
 - Efficient utilization of multicore/multiprocessor systems as the kernel can schedule different threads on different cores.
 - If one kernel-level thread is blocked (e.g., for I/O), other threads can continue execution.

Process Synchronization

Process synchronization is a fundamental concept in operating systems that ensures coordinated and safe access to shared resources by multiple concurrent processes. It involves using synchronization mechanisms to prevent conflicts, maintain data consistency, and avoid race conditions when processes access shared data or critical sections of code.

Importance of Process Synchronization:

- **Data Integrity:** Ensures that shared data remains consistent and does not get corrupted due to concurrent access.
- **Orderly Execution:** Ensures that processes execute in a controlled and expected order, especially when interacting with shared resources.
- **Prevention of Deadlocks and Race Conditions:** Helps avoid situations where processes are stuck waiting for each other, leading to a system halt or incorrect behavior.